

README — compute_gene_lengths.py

Non-overlapping exon length per gene from GTF (Snakemake-friendly)

Language	Python 3
Primary Dependencies	pandas, gzip (stdlib)
Fits	Snakemake workflows

Purpose

This README documents a Python utility that parses a GTF file, collects exon intervals per gene, merges overlapping exon segments, and computes the total non-overlapping exon length for each gene. The output is a tidy, tab-separated table with two columns: `gene_id` and `length`. The script is designed to fit into Snakemake workflows but can be adapted for standalone use.

Key Features

- Handles both compressed (.gtf.gz) and uncompressed GTF inputs automatically.
- Filters for feature type == 'exon' only.
- Merges overlapping exon intervals per gene to prevent double-counting.
- Outputs a simple TSV with header: `gene_id length`.
- Lightweight dependencies (Pandas only) and fast single-pass parsing.

Requirements

- Python 3.8+
- pandas (for building the final table)
- gzip (standard library) for .gz handling

Inputs

- GTF annotation file (e.g., GENCODE or RefSeq).
- May be plain text (e.g., `Homo_sapiens.GRCh38.107.gtf`) or gzip-compressed (.gtf.gz).

Outputs

- TSV file with two columns:
- `gene_id` — the gene identifier parsed from the GTF.
- `length` — total non-overlapping exon length (sum of merged exon intervals).

How It Works

- 1) Streams through the GTF, skipping comment lines (#).
- 2) Keeps only rows where feature type is 'exon'.
- 3) Parses start/end coordinates and attributes (extracts gene_id).
- 4) Collects exon intervals per gene in memory (as sets of (start, end)).
- 5) Merges overlapping intervals per gene (greedy, sorted by start).
- 6) Sums merged lengths (end – start + 1) to get a per-gene total.
- 7) Writes a TSV with gene_id and length.

Code Snippet (core API)

Snakemake Integration

```
rule compute_gene_lengths:
    input:
        gtf="path/to/annotation.gtf.gz"
    output:
        "results/gene_lengths.tsv"
    conda:
        "envs/py.yaml"
    script:
        "scripts/compute_gene_lengths.py"
```

- The script expects Snakemake to provide paths via `snakemake.input.gtf` and `snakemake.output[0]`.
- Example rule:

Troubleshooting

- Q: Output lengths look too small?
- A: Ensure your GTF contains exon features (feature column == 'exon').
- Q: Memory usage too high for very large GTFs?
- A: Consider sharding by chromosome or filtering upstream, then concatenate.
- Q: Mixed gene_id formats?
- A: Confirm the attribute key is 'gene_id' in your GTF (common for GENCODE/Ensembl).

Notes

- Exon length here measures annotated exonic span, not transcript-specific length.
- If you need transcript-level effective lengths, use a dedicated tool (e.g., tximport/salmon outputs).
- Coordinate system: GTF uses 1-based inclusive coordinates; lengths are computed as end – start + 1.

```
# Parse & compute (inside Snakemake rule) gene_lengths_df = parse_gtf(gtf_file)
gene_lengths_df.index.name = "gene_id" gene_lengths_df.to_csv(output_file,
sep="\t")
```

This document describes the expected behavior and integration points; adapt paths/IO to your project structure as needed.